RecSysOps: Best Practices for Operating a Large-Scale Recommender System

Mohammad Saberian
Justin Basilico
esaberian@netflix.com
jbasilico@netflix.com
Netflix, Inc.
Los Gatos, CA, USA

ABSTRACT

Ensuring the health of a modern large-scale recommendation system is a very challenging problem. To address this, we need to put in place proper logging, sophisticated exploration policies, develop ML-interpretability tools or even train new ML models to predict/detect issues of the main production model. In this talk, we shine a light on this less-discussed but important area and share some of the best practices, called RecSysOps, that we've learned while operating our increasingly complex recommender systems at Netflix. RecSysOps is a set of best practices for identifying issues and gaps as well as diagnosing and resolving them in a large-scale machine-learned recommender system. RecSysOps helped us to 1) reduce production issues and 2) increase recommendation quality by identifying areas of improvement and 3) make it possible to bring new innovations faster to our members by enabling us to spend more of our time on new innovations and less on debugging and firefighting issues.

KEYWORDS

Recommender Systems, RecSycOps, error detection, error prediction, model diagnostic, model explainability

ACM Reference Format:
Mohammad Saberian and Justin Basilico. 2021. RecSysOps: Best Practices for Operating a Large-Scale Recommender System. In *Fifteenth ACM Conference on Recommender Systems (RecSys '21), September 27-October 1, 2021, Amsterdam, Netherlands.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3460231.3474620

1 RECSYSOPS

Everyday at Netflix we serve personalized recommendations to our members. To do this, our system uses a diverse set of recommender models with different purposes and architectures [2]. We strive to constantly improve our member experience while making sure that our current system is healthy and serving the best recommendations. However, ensuring the health of a modern large-scale recommendation system is a very challenging problem because of increasing complexity of the ML models, e.g. ensembles, sophisticated deep neural networks, many data sources and many feature engineering components. These challenges are compounded by the sheer magnitude of number of users as well as number of items, which makes it challenging to ensure that all segments of users and items are getting optimal recommendations in the system. To address these challenges, we need to put in place proper logging, sophisticated exploration policies, develop ML-interpretability tools or even train new ML models to predict/detect issues of the main production model. In this talk, we shine a light on this less-discussed but important area and share some of the best practices, called RecSysOps, that we've learned while operating our increasingly complex recommender systems. RecSysOps helped us to 1) reduce production issues and 2) increase recommendation quality by identifying areas of improvement and 3) make it possible to bring new innovations faster to our members by enabling us to spend more of our time on new innovations and less on debugging and firefighting issues.

Figure 1: Components of RecSysOps

More specifically, RecSysOps is a set of best practices for identifying issues and gaps as well as diagnosing and resolving them in a large-scale machine-learned recommender system. The main components of RecSysOps are presented in Figure 1 and include modules for detecting and predicting issues, modules for diagnosing issues, and mechanisms for quickly and reliably resolving an issue. Next we provide more details about each module.

1.1 Issue Detection

Fast and accurate detection of issues in software and ML models is the most critical part of RecSysOps. For the software parts we can use software engineering best practices such as unit tests or continuous integration, and their ML equivalents such as regular, automated retraining and integration tests [1]. These, however, are not sufficient for complex ML models with many data dependencies in a dynamic high throughput environment. For the data processing steps we can include checks on absolute statistical properties of the data such as input/output counts. We can also check the relative values of such statistics against historic values or other pipelines,

RecSys '21, September 27-October 1, 2021, Amsterdam, Netherlands

ACM ISBN 978-1-4503-8458-2/21/09.
https://doi.org/10.1145/3460231.3474620

e.g. yesterday's run. For the health of model training steps, we can include checks on the absolute value of model metrics, such as NDCG, or checks on relative value of those metrics with respect to yesterday's model or a much simpler less error prune model, such as a popularity-based or random recommender.

In some cases while all of the code is working flawlessly and as expected, the system may still not serve optimal recommendations for some users or titles. Detecting these suboptimal system behaviors is a very challenging problem by itself and may require training a dedicated ML model for this purpose. The main challenge for building such a model is coming up with training datasets that are not subject to biases from the main production model. This can be achieved by adding additional sources of recommendations, for example (if feasible) through some stochasticity within the production system.

More generally, any time a user chose an item that was not ranked high by the production model, it is an indication of suboptimal recommendations. It is very useful to understand the root cause of such suboptimal recommendations. These can be rooted in wrong modeling assumptions, insufficient input data or biases and errors in the training datasets. It is useful to collect these cases, visualize and analyse them on a regular basis to be able to find gaps in the systems.

1.2 Issue Prediction

The environment of recommender systems platforms are highly dynamic and changes every day. In the context of Netflix, one such change could be the addition of new TV shows or movies to the catalog. Before launching a new show we need to make sure that the models are able to properly score the show and recommend it to the right set of users. This requires simulating and predicting future recommendations, i.e. training a model to predict future behavior of the production models and flag any unexpected behaviors.

1.3 Issue Diagnosis

The first step in diagnosing ML related issues is to reproduce them in isolation. To enable this, we need to put in place proper data logging in advance. For example, in the case of a real-time recommendation model, we should log the model's input and output for a sample of users on a regular basis. In addition, the sampling method needs to ensure that we have proper support for all segments of users, e.g. small countries.

Next we need to understand if an issue is related to the model or its inputs. Diagnosing issues related to the input data starts by identifying features or inputs that contribute to the unexpected model's output, e.g. low score. For this we can use anomaly detection methods to identify if a feature value is abnormal, or we can use model interpretation tools such as LIME [4], SHAP [3] or feature sensitivity analysis to identify potential root causes for the unexpected output and investigate them.

Finally if all inputs are as expected, we need to investigate the model and find the root cause of its output. For this we need tools to visualize internal components of a model, e.g. internal layers of a deep network, structure of decision trees, etc.

1.4 Issue Resolution

Depending on time-sensitivity of an issue and its root cause, we can take short or long term actions. For urgent matters about ML models, we need to have mechanisms in place to quickly deploy a new model or augment/alter the model's output quickly as a hotfix. This is similar to software management where we have to release hotfixes or patches. In addition, once a pattern of issues is identified, after implementing a fix for it we need to set up lower level tests, e.g. unit tests, to prevent regressions and catch them faster in future. There are, however, cases where some issues can happen for reasons out of our control, e.g. some timeouts or corrupted data. In this case we need to bake those situations into our modeling assumptions and make the models robust against them. For example assume that a feature value might become missing in an unpredictable fashion. To make a model robust against this, we set values of that feature to be missing for a random set of samples during the training. Similar to dropout [5], this will enable the model to have support for scoring samples whose feature values are missing.

1.5 Acknowledgements

We are very thankful to our collaborators - Ragavika Tarigopula, Elmar Nubbemeyer, Yves Raimond, Molly Jackman, Varun Khaitan, Jaewook Yu, Sura Elamurugu, Jen Guenther, Michelle Kislak, Danielle Rosenberg, Pablo Delgado, Gabriel Ng and many more.

2 SPEAKER BIO

Mohammad (Ehsan) Saberian is a Senior Machine Learning Researcher at Netflix working on problems related to ranking and large-scale software engineering. His work spans various research areas such as Boosting, Deep Neural Networks and Model Interpretability. Previously he was a research scientist at Yahoo! Labs working on computer vision and natural language processing problems. He holds a PhD in machine learning and computer vision from University of California San Diego.

REFERENCES

[1] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. In *Proceedings of IEEE Big Data.*

[2] Carlos A. Gomez-Uribe and Neil Hunt. 2016. The Netflix Recommender System: Algorithms, Business Value, and Innovation. *ACM Trans. Manag. Inf. Syst.* 6, 4 (2016), 13:1–13:19. http://dblp.uni-trier.de/db/journals/tmis/tmis6.html#Gomez-UribeH16

[3] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.). Curran Associates, Inc., 4765–4774. http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf

[4] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) *(KDD '16)*. Association for Computing Machinery, New York, NY, USA, 1135–1144. https://doi.org/10.1145/2939672.2939778

[5] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15, 56 (2014), 1929–1958. http://jmlr.org/papers/v15/srivastava14a.html